

Sorting, Searching & Lists (Algo + C)

Exercises with solutions in pseudo-code and C

Exercise 1 — Sequential search (index or -1)

Task. Read n , an array of integers, and a key x . Return the index of the first occurrence or -1 .

Exercise 2 — Binary search (sorted array)

Task. On a strictly increasing sorted array, search for x .

Exercise 3 — Min & Max in one pass

Task. Find the minimum and maximum values of an integer array in a single pass.

Exercise 4 — Selection sort

Task. Sort an array of integers using selection sort.

Exercise 5 — Insertion sort

Task. Implement insertion sort.

Exercise 6 — Bubble sort (optimized)

Task. Bubble sort with early stopping if no swap occurs during a pass.

Exercise 7 — Merge two sorted arrays

Task. Merge sorted arrays A and B into a sorted array C .

Exercise 8 — Merge sort

Task. Implement recursive merge sort (stable).

Exercise 9 — Quicksort (Lomuto partition)

Task. Implement quicksort using Lomuto's partition scheme (pivot at the end).

Exercise 10 — Counting sort (integers in $[0..k]$)

Task. Count occurrences and reconstruct the sorted array.

Exercise 11 — Radix sort LSD (base 10, integers ≥ 0)

Task. Apply stable counting sort digit by digit (least significant first).

Exercise 12 — Heap sort

Task. Build a max-heap then extract the elements.

Exercise 13 — Quickselect (k -th smallest, 1-indexed)

Task. Find the k -th order statistic without fully sorting the array.

Exercise 14 — Remove duplicates from a sorted array (in-place)

Task. Compact the array so that each value appears once. Return the new size.

Exercise 15 — Target sum (two-pointer)

Task. Determine whether there exist $i < j$ such that $a_i + a_j = S$.

Exercise 16 — Linked list : insert at head & tail

Task. Implement `push_front` and `push_back`, then print the list.

Exercise 17 — Linked list : remove first occurrence

Task. Implement `remove_first(&H, x)`.

Exercise 18 — Linked list : iterative reversal

Task. Implement `reverse(&H)` to reverse the list.

Exercise 19 — Merge two sorted linked lists

Task. Merge two sorted singly linked lists into one sorted list (ascending).

Exercise 20 — Cycle detection (Floyd)

Task. Detect whether a cycle exists in a list using the tortoise-hare algorithm.
